DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 06
Checkpoint e revisão para a 1ª VA

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Esta apostila fecha o primeiro bloco do semestre. Ela tem duas funções. A primeira é revisar, em tabelas e mapas, tudo o que as semanas 1 a 5 construíram: REST, semântica HTTP, camadas, injeção de dependência, ORM, transações, migrações, DTO, ProblemDetail e OpenAPI. A segunda é preparar o checkpoint do projeto, que é a tag `etapa-2` no repositório da equipe. A 1ª VA acontece na semana 7 e cobra o *porquê* de cada decisão, não o nome de cada anotação. Por isso a seção de exercícios é longa e no estilo da prova.

Conteúdo

1. Objetivos da semana
2. Como revisar: o mapa das semanas 1 a 5
3. REST e suas restrições
4. Semântica HTTP: métodos e status
5. O domínio: a máquina de estados
6. IoC, injeção de dependência e camadas
7. ORM e o ciclo de vida da entidade
8. Transações
9. Migrações com Flyway
10. Por que não H2
11. DTO, ProblemDetail e OpenAPI
12. Quadro-resumo e onde cada conceito mora no código
13. O checkpoint: o que precisa estar no repositório
14. Construindo com IA
15. Glossário
16. Exercícios estilo prova
17. Para estudar

1. Objetivos da semana

↑ conteúdo

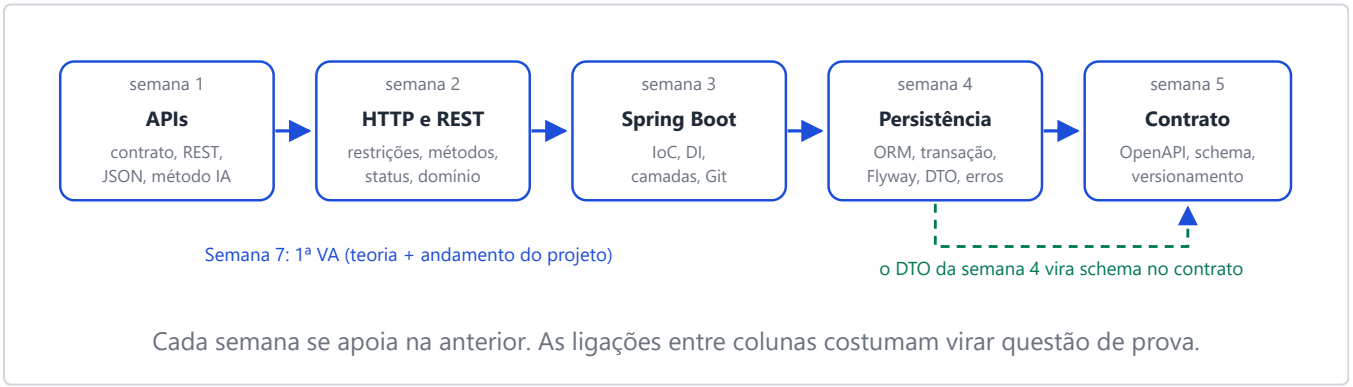
Ao final desta semana você deve ser capaz de:

1. escolher método, URI e status code para um cenário do ServiceHub e justificar a escolha numa frase;
2. explicar o papel de cada camada do Spring Boot e onde ficam as regras de negócio e a transação;
3. descrever o ciclo de vida de uma entidade JPA e prever o que vai ao banco num método `@Transactional`;
4. relacionar Flyway, `ddl-auto=validate`, DTO, ProblemDetail e OpenAPI ao problema que cada um resolve;
5. conferir o repositório da equipe contra o checklist do checkpoint e publicar a tag anotada `etapa-2`, verificada num clone limpo.

2. Como revisar: o mapa das semanas 1 a 5

↑ conteúdo

As cinco primeiras semanas não são cinco assuntos soltos. Cada uma apoia a seguinte: o contrato da Semana 1 vira rotas HTTP na Semana 2; as rotas viram controladores na Semana 3; os controladores ganham banco, DTOs e erros padronizados na Semana 4; e os DTOs viram schemas do contrato OpenAPI na Semana 5. Quem estuda os temas isolados decora; quem enxerga a ligação entende.



Cada semana se apoia na anterior. As ligações entre colunas costumam virar questão de prova.

O que a prova pede

A 1ª VA tem duas partes: a teoria das semanas 1 a 5 e a apresentação do andamento do projeto. Nas questões teóricas, uma resposta boa tem três elementos: **o quê** (o status, a camada, a anotação), **por quê** (o conceito que justifica) e **onde** (um caso do ServiceHub). Compare "409, conflito", que não mostra entendimento, com "409, porque o e-mail já existe (RN01): a requisição está bem formada, mas conflita com o estado atual do recurso".

TÉCNICA DE ESTUDO

Use o quadro-resumo da seção 12: tampe a coluna do meio e explique cada linha em voz alta. Se você só lembra o nome da anotação e não o problema que ela resolve, ainda não entendeu o conceito.

3. REST e suas restrições

↑ conteúdo

REST é um estilo arquitetural definido por Roy Fielding na sua tese de doutorado, em 2000. Um estilo arquitetural não é uma biblioteca nem um protocolo: é um conjunto de **restrições**. Quem respeita as restrições ganha certas propriedades, como escalabilidade e independência entre cliente e servidor. Uma API **RESTful** é a que respeita essas restrições, e não simplesmente a que usa JSON.

Restrição	O que exige	O que se ganha
Cliente-servidor	Interface e dados separados por uma API.	Cada lado evolui sozinho.
Sem estado	Cada requisição traz tudo de que precisa, inclusive a credencial (no ServiceHub, o token).	Qualquer instância atende qualquer requisição.
Cache	A resposta diz se pode ser guardada e por quanto tempo.	Menos idas ao servidor.
Interface uniforme	Recursos identificados por URI, manipulados por representações, com métodos e status padronizados.	Quem conhece HTTP já sabe usar a API.
Sistema em camadas	O cliente não sabe se fala com o servidor final ou com um intermediário.	Balanceador, proxy e gateway no meio, sem mudar o cliente.
Código sob demanda	Opcional: o servidor envia código executável.	Raramente usada em APIs.

A restrição mais cobrada: sem estado

Imagine uma loja virtual com duas instâncias da API atrás de um balanceador. Se a instância A guarda na memória "o usuário 7 está logado", a próxima requisição desse usuário pode cair na instância B, que não sabe de nada e o trata como desconhecido. A saída é cada requisição carregar a própria prova de identidade. No ServiceHub, essa prova é o token JWT enviado no cabeçalho `Authorization` (tema da Semana 09).

CONFUSÃO COMUM

"Usa HTTP e JSON, então é REST." Uma API com `POST /getUser` ou `GET /deleteService?id=3` usa HTTP só como transporte: o verbo está na URI e o método perdeu o significado. Isso quebra a interface uniforme.

URIs: substantivos, não verbos

A URI identifica o recurso; o método diz a ação. Coleções vão no plural, em minúsculas, com hífen entre palavras (`/service-requests`); o identificador vem depois da coleção (`/services/12`); filtros, ordenação e paginação vão na query string; a versão aparece no caminho desde o primeiro endpoint (`/api/v1`). Uma mudança de estado específica, como a da ordem, vira um sub-recurso: `PATCH /api/v1/service-requests/31/status`.

4. Semântica HTTP: métodos e status

↑ conteúdo

Uma mensagem HTTP tem três partes que dizem coisas diferentes. O **método** diz a intenção; o **status code** diz o resultado; os **cabeçalhos** dizem o formato e os metadados. Dois cabeçalhos aparecem em quase toda prova: `Content-Type` descreve o corpo que está sendo enviado, e `Accept` pede o formato da resposta.

Requisição

```
POST /api/v1/services HTTP/1.1
Content-Type: application/json
Accept: application/json

{ "title": "Pintura", ... }
```

Resposta

```
HTTP/1.1 201 Created
Location: /api/v1/services/12
Content-Type: application/json

{ "id": 12, "title": "Pintura", ... }
```

Métodos: seguro e idempotente

Um método é **seguro** quando não altera o estado do servidor. É **idempotente** quando repetir a mesma chamada deixa o servidor no mesmo estado que uma chamada só, mesmo que a resposta mude. Essas propriedades importam porque clientes, proxies e bibliotecas repetem requisições automaticamente quando a rede falha: repetir um GET ou um PUT é inofensivo; repetir um POST pode criar dois registros.

Método	Exemplo no ServiceHub	Seguro	Idempotente
GET	`GET /api/v1/services?page=0&size=20`	Sim	Sim
POST	`POST /api/v1/services` → 201 + Location	Não	Não
PUT	`PUT /api/v1/services/12` (substitui a representação)	Não	Sim
PATCH	`PATCH /api/v1/service-requests/31/status`	Não	Não garantido
DELETE	`DELETE /api/v1/services/12` → 204	Não	Sim

PERGUNTA CLÁSSICA

O segundo `DELETE` do mesmo serviço é idempotente mesmo respondendo diferente da primeira vez? Sim. Idempotência fala do estado final do servidor, e não da resposta. No ServiceHub o DELETE é exclusão lógica (ADR-0002): o registro continua no banco com `active=false`, e o estado final é o mesmo depois de uma ou de várias chamadas.

Status: cada cenário tem um só

As famílias de status respondem "de quem é a culpa". **2xx** é sucesso. **3xx** é redirecionamento. **4xx** é erro do cliente: ele pode corrigir o pedido e tentar de novo. **5xx** é erro do servidor e nunca é uma resposta planejada: um 500 significa que faltou tratar um caso.

Cenário no ServiceHub	Resposta
Serviço criado	201 Created + `Location`
Listagem de serviços	200 OK, página com `content`
Serviço desativado (exclusão lógica)	204 No Content
Preço ausente no corpo	400 com `errors[]`
`GET /api/v1/services/999`	404 Not Found
E-mail já cadastrado (RN01)	409 Conflict
Ordem COMPLETED volta para ACCEPTED	409 `invalid-transition`
Corpo enviado como `text/plain`	415 Unsupported Media Type

A CONFUSÃO MAIS COMUM DA PROVA: 400 × 409

400 diz que o pedido está malformado: falta campo, o tipo está errado, o valor está fora do formato. **409** diz que o pedido é válido, mas conflita com o estado atual do recurso: o e-mail já existe, a transição não é permitida a partir do status atual. Teste mental: "se o banco estivesse vazio, este pedido daria certo?" Se sim, o problema é de estado, e a resposta é 409.

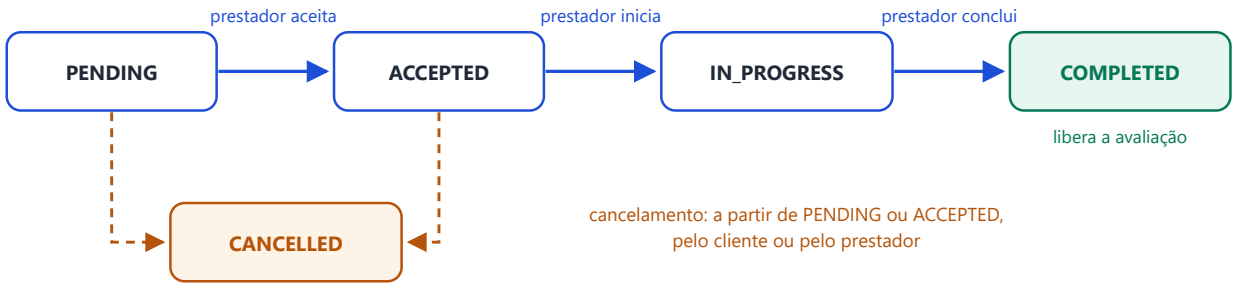
Dois status voltam com força na Semana 08: **401** significa "não sei quem você é" (falta autenticação ou o token é inválido); **403** significa "sei quem você é, e você não pode fazer isto".

5. O domínio: a máquina de estados da ordem

↑ conteúdo

Uma **máquina de estados** descreve os estados possíveis de um objeto e as transições permitidas entre eles. Ela aparece em qualquer domínio com um processo: um pedido de compra (aguardando pagamento, pago, enviado, entregue), uma matrícula, um chamado de suporte.

Modelar o processo como máquina de estados torna explícito o que é proibido, e o proibido vira regra testável.



Estados da ordem de serviço no Mapa de Construção (RN08). Qualquer seta que não está no desenho não existe.

A tabela de transições produz dois erros diferentes, e essa distinção cai em prova:

Situação	Status	Por quê
COMPLETED → ACCEPTED, pedido por qualquer um	409	A transição não existe na tabela, para ninguém. É conflito com o estado atual.
PENDING → ACCEPTED, pedido pelo cliente	403	A transição existe, mas não pertence ao papel de quem pediu.

O QUE A IA NÃO SABE SOZINHA

As regras vêm do Mapa, não do código gerado. Um assistente que nunca leu a seção 4 do Mapa vai inventar uma tabela plausível, por exemplo deixando o cliente concluir a ordem. A tabela de transições é o tipo de conhecimento de domínio que só a equipe pode especificar.

6. IoC, injeção de dependência e camadas

↑ conteúdo

Os termos

- **Inversão de controle (IoC):** quem cria e liga os objetos é o framework, e não o seu código. Você declara as peças; o contêiner monta o sistema.
- **Injeção de dependência (DI):** a dependência chega de fora, de preferência pelo construtor.
- **Bean:** objeto criado e gerenciado pelo contêiner do Spring.

Por que isso existe: uma classe que faz `new RepositorioPostgres()` por dentro fica presa àquela implementação. Uma classe que recebe o repositório pelo construtor depende só de um tipo, e o teste pode entregar um dublê no lugar do repositório real. No ServiceHub, os serviços recebem todas as dependências no construtor:

```
@Service
@Transactional
public class ServiceRequestService {
    private final ServiceRequestRepository requestRepository;
    private final CurrentUser currentUser;
    // ... mais três dependências, todas recebidas no construtor (resumido)
}
```

Injeção no campo

```
@Autowired
private ServicoRepository repo;
```

O campo não pode ser `final`, a dependência fica escondida e o teste sem Spring precisa de reflexão para preencher o campo.

Injeção pelo construtor

```
private final ServicoRepository repo;
ServicoService(ServicoRepository repo) {
  this.repo = repo;
}
```

Campos `final`, dependências explícitas na assinatura e teste com um simples `new ServicoService(dubleDoRepo)`.

Camadas: cada uma tem um papel e um limite

Camada	Anotações	Faz	Nunca
Controller	`@RestController`, `@Valid`	Rota, DTO, status code	Regra de negócio
Service	`@Service`, `@Transactional`	Regras RN, monta o DTO de resposta	Devolver entidade JPA
Repository	`JpaRepository`	Consultas parametrizadas	Decidir regra
Entidade	`@Entity`, `@Table`	Espelha a tabela	Sair na resposta HTTP

Pergunta típica: onde fica a regra "ninguém contrata o próprio serviço" (RN06)? No Service. Ela precisa do usuário atual e de dados do banco, e precisa valer para qualquer chamador, seja um controller, um teste ou uma tarefa agendada. Se estivesse no controller, outro ponto de entrada poderia pular a regra.

PARA LEMBRAR

Com `spring.jpa.open-in-view=false`, a sessão do banco fecha quando o método do Service termina. Por isso o DTO de resposta é montado dentro do método `@Transactional` do Service, e não no controller.

7. ORM e o ciclo de vida da entidade

↑ conteúdo

ORM (*Object-Relational Mapping*, mapeamento objeto-relacional) traduz entre o modelo de objetos do Java e o modelo relacional do banco. Os dois modelos pensam de forma diferente: objetos têm herança, identidade por referência e navegação de um objeto a outro; tabelas têm linhas, chaves e junções. Essa diferença tem nome, **incompatibilidade de impedância objeto-relacional**, e é o problema que o ORM resolve. Sem ele, cada consulta é SQL escrito à mão e cada linha é convertida em objeto campo a campo.

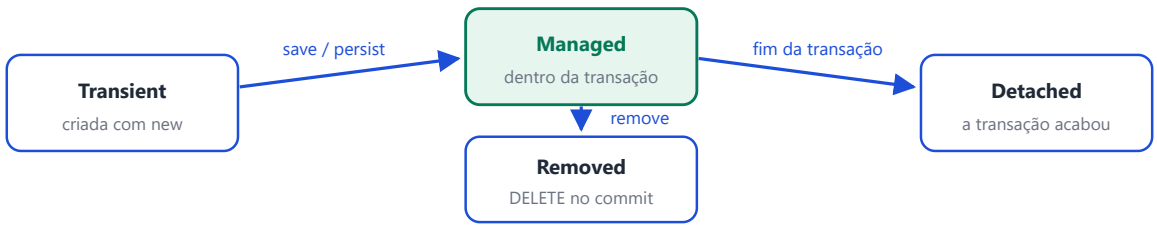
JPA (*Jakarta Persistence*) é a especificação, o contrato. **Hibernate** é a implementação que o Spring Boot usa. São coisas diferentes, como uma interface Java e a classe que a implementa.

No Java	No banco	No ServiceHub
Classe `@Entity`	Tabela	`ServiceOffering` ↔ `services`
Atributo	Coluna	`price` ↔ `price NUMERIC`
Objeto	Linha	Um serviço publicado
Referência a outro objeto	Chave estrangeira	`provider` ↔ `provider_id`
Enum	Texto, com ou sem CHECK	`@Enumerated(STRING)` ↔ `VARCHAR`

O CUSTO DO ORM

Navegar por referências sem cuidado gera o problema **N+1**: uma consulta para trazer N serviços e mais uma consulta por serviço para trazer o prestador de cada um. A ferramenta esconde o SQL, mas não o elimina.

Os quatro estados de uma entidade



Ciclo de vida da entidade JPA. Removed parte de managed; uma entidade detached precisa voltar a ser managed antes de ser removida.

O **contexto de persistência** é o "caderno" em que o Hibernate anota as entidades managed. No commit, ele compara cada entidade com o que leu do banco; o que mudou vira `UPDATE`. Esse mecanismo se chama **dirty checking**. Consequência prática: uma entidade managed alterada por um setter é gravada no commit mesmo sem chamar `save`. Fora da transação, a mesma alteração fica só no objeto em memória e se perde.

No ServiceHub a exclusão é lógica, então o fluxo real de um DELETE é: *managed* → `setActive(false)` → `UPDATE` no commit. E, com `open-in-view=false`, acessar uma associação *lazy* numa entidade detached lança `LazyInitializationException`.

8. Transações

↑ conteúdo

Uma **transação** agrupa operações no banco para que todas aconteçam ou nenhuma aconteça. O exemplo clássico é a transferência bancária: debitar de uma conta e creditar na outra. Se o sistema cair entre as duas operações, o dinheiro não pode sumir. As garantias de uma transação formam a sigla **ACID**:

Garantia	Significado
Atomicidade	Falhou uma parte, desfaz-se tudo.
Consistência	As restrições do banco (NOT NULL, CHECK, FK) valem ao fim da transação.
Isolamento	Transações simultâneas não enxergam o trabalho pela metade uma da outra.
Durabilidade	O que foi confirmado não se perde, mesmo com queda do servidor.

No ServiceHub, desativar um usuário também desativa os serviços dele (RN11). Se a segunda parte falhar, a primeira precisa ser desfeita: isso é atomicidade.

`@Transactional`	Comportamento padrão no Spring
Onde colocar	No Service, onde a regra de negócio começa e termina. A unidade de trabalho é a regra, que pode tocar vários repositórios.
Rollback	Em exceções não verificadas (`RuntimeException` e `Error`). Exceções verificadas não desfazem a transação, a menos que se configure `rollbackFor`.
`readOnly = true`	Para consultas: avisa que não haverá escrita e dispensa o dirty checking.

PEGADINHA

`@Transactional` funciona por meio de um proxy que o Spring coloca em volta do bean. Uma chamada de um método para outro *dentro da mesma classe* não passa pelo proxy, então a anotação do segundo método é ignorada.

9. Migrações com Flyway

↑ conteúdo

Uma **migração** é um script versionado que leva o schema do banco de uma versão para a próxima. A ideia vale para qualquer linguagem: o schema passa a ser código, guardado no Git, aplicado na mesma ordem em todas as máquinas. Sem migrações, cada colega tem um banco ligeiramente diferente, e o erro aparece só no ambiente de outra pessoa.

O **Flyway** aplica os arquivos `V1__descricao.sql`, `V2__...` em ordem e registra cada um numa tabela de histórico, com um **checksum** (uma soma de verificação do conteúdo). No ServiceHub, o Flyway escreve o schema e o Hibernate só confere:

```
-- V5__Create_service_requests_table.sql (resumido)
CREATE TABLE service_requests (
    id           BIGSERIAL PRIMARY KEY,
    service_id   BIGINT NOT NULL REFERENCES services(id),
    client_id    BIGINT NOT NULL REFERENCES users(id),
    status       VARCHAR(30) NOT NULL DEFAULT 'PENDING',
    total_price  NUMERIC(10,2) NOT NULL,
    CONSTRAINT chk_status CHECK (status IN
        ('PENDING','ACCEPTED','IN_PROGRESS','COMPLETED','CANCELLED'))
);
```

```
# application.properties
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
```

Regra	Por quê
Toda mudança de entidade vem com uma nova migração	Com `validate`, o Hibernate não cria nem altera nada: se faltar coluna, a aplicação não sobe, e o erro diz qual.
Migração aplicada nunca é editada	O checksum muda e o Flyway recusa iniciar. Corrige-se com uma nova versão; no projeto de referência, a V6 removeu um índice redundante criado antes.
CHECK no banco mesmo com enum no Java	O banco é a última linha de defesa contra qualquer cliente que escreva nele: um script, outro sistema, uma migração futura.

10. Por que não H2: Postgres, Flyway e Testcontainers

↑ conteúdo

O **H2** é um banco em memória escrito em Java, muito usado em tutoriais porque dispensa instalação. O problema é que ele fala outro dialeto de SQL. As migrações do ServiceHub usam recursos do PostgreSQL, como `BIGSERIAL`, `NUMERIC(10,2)`, `CHECK (status IN (...))` e `ALTER TABLE ... DROP CONSTRAINT`. Com H2, ou o Flyway falha, ou é preciso desligá-lo e usar `ddl-auto=create-drop`, e aí o teste roda contra um schema que não existe em produção.

Critério	H2 em memória	Postgres 16 + Flyway
Migrações	Outro dialeto: tipos, CHECK e ALTER podem falhar ou se comportar diferente	As mesmas V1…Vn de produção
Schema	Tentação de `create-drop` ou `update`	`ddl-auto=validate`
Erro de SQL	Aparece no deploy	Aparece no teste

Critério	H2 em memória	Postgres 16 + Flyway
Nos testes	"Zero configuração"	Testcontainers com `postgres:16-alpine`

O **Testcontainers** é uma biblioteca que, durante o teste, sobe um container Docker real e o descarta no fim. No ServiceHub, uma classe de configuração entrega o container ao Spring, e a anotação `@ServiceConnection` preenche URL, usuário e senha do datasource. O Flyway então aplica as mesmas migrações de produção a cada execução:

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {
    @Bean
    @ServiceConnection
    PostgreSQLContainer postgresContainer() {
        return new PostgreSQLContainer("postgres:16-alpine");
    }
}
```

O preço é ter o Docker ligado ao rodar os testes. A decisão e as consequências estão registradas no **ADR-0005** do projeto de referência (`docs/adr/0005-testcontainers-em-vez-de-h2.md`). Os testes de integração com banco são tema da Semana 10; por ora, basta saber justificar a escolha.

11. DTO, ProblemDetail e OpenAPI

↑ conteúdo

DTO: a forma dos dados na fronteira

DTO (*Data Transfer Object*) é um objeto simples, sem regra, que carrega dados entre camadas ou sistemas. Martin Fowler descreve o padrão no catálogo de padrões de aplicações corporativas. Numa API, o DTO é o corpo que entra e o corpo que sai.

Aspecto	Entidade	DTO
Representa	Uma linha da tabela	Um caso de uso da API
Muda quando	O banco muda	O contrato muda
Campos	Todos, inclusive `passwordHash`	Só os que o cliente pode ver ou enviar
Validação	Restrições do banco	Bean Validation (`@NotBlank`, `@Size`)

O DTO evita dois riscos: **vazar** dados internos na resposta e **aceitar** campos que o cliente não deveria definir, problema conhecido como *mass assignment* e aprofundado na Semana 08. No ServiceHub, o DTO de criação de ordem não tem `clientId`, `status` nem preço; a API define os três:

```
@Schema(description = "Abertura de ordem de serviço. Cliente, status e preço são definidos
pela API.")
public record ServiceRequestCreateDTO(
        @NotNull(message = "O serviço é obrigatório") @Positive Long serviceId,
        @Future(message = "A data agendada deve estar no futuro") LocalDateTime scheduledAt,
        @Size(max = 1000) String notes) { }   // resumido
```

ProblemDetail: todo erro no mesmo formato

A **RFC 9457** (*Problem Details for HTTP APIs*) define um formato JSON padrão para erros, com o tipo de mídia `application/problem+json`. O Spring tem suporte nativo pela classe `ProblemDetail`. O campo `type` é o identificador estável do erro: o cliente da API programa contra ele, e não contra o texto de `detail`, que pode mudar.

Resposta 400

```
{
  "type":
"urn:servicehub:problem:validation-error",
  "title": "Dados inválidos",
  "status": 400,
  "detail": "Um ou mais campos estão
inválidos.",
  "instance": "/api/v1/services",
  "errors": [
    { "field": "price",
      "message": "O preço é obrigatório" }
  ]
}
```

Um ponto central de tradução

O `GlobalExceptionHandler`, anotado com `@RestControllerAdvice`, converte:

- `ResourceNotFoundException` → 404
- `EmailAlreadyExistsException` → 409
- `BusinessRuleException` → 409
- falha de `@Valid` → 400 com `errors`
- qualquer outra → 500, sem stack trace

ERRO COMUM EM CÓDIGO GERADO POR IA

Lançar `RuntimeException("preço inválido")` para uma entrada errada. Ninguém trata essa exceção genérica, e o cliente recebe um 500 em vez de um 400 com o campo apontado.

OpenAPI: o contrato que se lê sem abrir o código

A **OpenAPI** é uma especificação para descrever APIs HTTP num documento JSON ou YAML: rotas, parâmetros, corpos, respostas e schemas. O **Springdoc** gera esse documento a partir do código; a linha 3.0.x é a que funciona com o Spring Boot 4. No projeto de referência, a interface Swagger UI fica em `/docs` e o documento em `/api-docs/v1`.

Anotação	Uso bem feito
`@Tag`, `@Operation`	Um *summary* que diz o que a rota faz no domínio ("Abrir ordem de serviço"), nunca "update resource".
`@ApiResponse`	Um por status possível; os de erro com o schema `ProblemDetail`.

Anotação	Uso bem feito
`@Schema` nos DTOs	Exemplos reais ("Instalação elétrica residencial"), não "string".

Uma confusão frequente: no documento OpenAPI, o campo `openapi` é a versão da *especificação* usada; o campo `info.version` é a versão do *seu contrato*. Teste rápido de qualidade: alguém de fora consegue chamar a rota certa só lendo o Swagger?

12. Quadro-resumo e onde cada conceito mora no código

↑ conteúdo

Conceito	Problema que resolve	No ServiceHub
REST sem estado	Escalar sem amarrar o cliente a um servidor	Token em cada requisição
IoC e DI	Classes acopladas e difíceis de testar	Construtor com campos `final`
Camadas	Regra espalhada por controller e SQL	Regras RN no Service
ORM (JPA)	Converter linhas em objetos à mão	`ServiceOffering` ↔ `services`
Transação	Metade da operação gravada	`@Transactional` no Service
Migração versionada	Bancos diferentes em cada máquina	Flyway V1…V9, `ddl-auto=validate`
Postgres nos testes	Teste verde e deploy quebrado	Testcontainers (ADR-0005)
DTO	Vazar campos ou aceitar campos indevidos	Records de entrada e saída
ProblemDetail	Cada erro num formato	`GlobalExceptionHandler`
OpenAPI	Consumidor adivinhando o contrato	`/docs` e `/api-docs/v1`

Onde procurar no projeto de referência

Na apresentação do checkpoint você vai mostrar uma classe de cada camada. No projeto de referência, os conceitos revisados ficam em: `controller/ServiceOfferingController.java` (rotas, `@Valid`, 201 com `Location`, paginação); `service/ServiceRequestService.java` (RN06–RN08, `@Transactional`, injeção pelo construtor); `model/RequestStatus.java` (máquina de estados como dados, ADR-0006); `exception/GlobalExceptionHandler.java` e `exception/ProblemTypes.java` (ProblemDetail e as URNs do campo `type`); `src/main/resources/db/migration/` (V1…V9); `application.properties` (`validate`, `open-in-view=false`, paginação 20/100); e `docs/adr/` (decisões registradas). Use a mesma organização no repositório da equipe: quem avalia deve achar cada conceito sem pedir ajuda.

13. O checkpoint: o que precisa estar no repositório

↑ conteúdo

No checkpoint vale o repositório, não a promessa. A avaliação parte do commit marcado pela tag `etapa-2`: o que não está nesse commit não foi entregue, mesmo que funcione na máquina de alguém. O checkpoint não é formalidade; é a chance de receber devolutiva antes da entrega final.

Item	O que precisa existir	Sem.
Banco por migração	Flyway com as tabelas do domínio; `ddl-auto=validate`.	4
Um CRUD completo	Em `/api/v1`, em camadas, com DTOs; `POST` devolve 201 e `Location`.	4
Listagem paginada	`Pageable` com padrão de 20 itens e máximo de 100.	4
Erros em ProblemDetail	`@RestControllerAdvice` responde 400, 404 e 409 no mesmo formato.	4
Contrato OpenAPI	Swagger UI com todas as rotas e os status de erro documentados.	5
Prova e decisões	Um `@WebMvcTest` passando, `docs/dominio.md` e ao menos um ADR.	2–3
README	Como subir o Postgres, como rodar a API e onde fica o Swagger.	3

A entrega é uma tag anotada

Uma **tag** é um nome fixo apontado para um commit. A tag **anotada** (`git tag -a`) é um objeto próprio no Git, com autor, data e mensagem; a tag leve é só um apelido. Para uma entrega, use a anotada: ela registra quem marcou e quando. Depois de marcar, a `main` continua evoluindo, e a avaliação olha exatamente o commit marcado.

```
git tag -a etapa-2 -m "Etapa 2: checkpoint da 1ª VA"
git push origin etapa-2

# teste final, em outra pasta
git clone <url-do-repo> conferencia
cd conferencia
git checkout etapa-2
./mvnw verify          # no Windows: mvnw.cmd verify
```

O **clone limpo** pega o clássico "na minha máquina funciona": um arquivo que nunca foi commitado, um segredo que só existe no seu `.env`, uma migração editada depois de aplicada no seu banco local. Se o `verify` passa numa pasta recém-clonada, com o Docker ligado, a entrega é reprodutível.

Antes de marcar

☐ O README diz como subir o Postgres e a API.

☐ Nenhum segredo commitado: `.env` está no `.gitignore`.

☐ Nenhuma migração já aplicada foi editada.

☐ `./mvnw verify` passa num clone limpo.

☐ `git show etapa-2 --stat` mostra o commit esperado.

Roteiro da apresentação: 8 a 10 minutos

1. **Contexto:** domínio, papéis e a jornada do cliente e do prestador.
2. **Swagger:** rotas e erros documentados.
3. **Ao vivo:** uma chamada que dá 201, uma que dá 400 e uma que dá 404.
4. **Código:** uma classe de cada camada.
5. **Decisão:** um ADR, com problema, decisão e efeito. É a parte que a IA não faz pela equipe.

LOGÍSTICA

Suba o banco e a API antes de começar. Erro de compilação ao vivo consome o tempo da equipe. Cada integrante deve conseguir falar de pelo menos uma etapa do roteiro.

Entrega da semana

- Tag `etapa-2` publicada no GitHub, num commit em que `./mvnw verify` passa.
- README com: como subir o Postgres, como rodar a API e onde fica o Swagger.
- `docs/revisao.md` com as respostas dos exercícios 1 e 2 da seção 16, com uma frase de justificativa em cada item.
- Um parágrafo: uma resposta errada da IA no estudo e como você descobriu o erro.

14. Construindo com IA

↑ conteúdo

Nesta semana não há funcionalidade nova para construir. O assistente tem dois usos: **estudar** para a prova e **revisar** o repositório antes da tag. Nos dois casos, a regra é a mesma: a IA não responde por você. Na prova não há IA; ela serve para treinar o raciocínio, não para guardar as respostas.

Movimento	Nesta semana
Especificar	Dê o escopo: semanas 1 a 5, o domínio ServiceHub, as regras do Mapa. Diga que quer perguntas, não respostas.
Gerar	"Crie 5 cenários do ServiceHub e pergunte o status HTTP. Só corrija depois que eu responder."
Revisar	Confira a correção da IA contra esta apostila, o Mapa e o seu código. Ela erra com a mesma confiança com que acerta.
Provar	Divergência entre você e a IA vira teste: um `@WebMvcTest` que espera 400 ou 409 decide quem está certo.

Prompt fraco e prompt bom

Fraco

```
me dá um resumo de tudo que cai na prova de API
```

A IA entrega um resumo genérico, que você lê sem pensar. Nada é treinado; e o resumo pode conter erros que você não tem como perceber.

Bom

Pede o papel de tutor, um tema de cada vez, cenários do ServiceHub, espera pela sua resposta antes de corrigir e cola as regras do Mapa para a IA não inventar outras. Veja o prompt completo abaixo.

Prompt de tutor (para estudar)

```
Contexto: sou aluno de Engenharia de Software e faço na próxima semana
a 1ª VA de Desenvolvimento de API Back-End. O projeto da disciplina é
a API ServiceHub (Java 21, Spring Boot 4.1, PostgreSQL 16, Flyway,
Springdoc 3.0.x). A prova cobre: restrições REST, métodos e status
HTTP, IoC/DI e camadas, JPA (ciclo de vida da entidade, transações),
Flyway, DTO, ProblemDetail (RFC 9457) e OpenAPI.

Papel: seja meu tutor. NÃO me dê a resposta antes de eu tentar.

Regras:
1. Faça UMA pergunta por vez, sempre com um cenário do ServiceHub.
2. Espere a minha resposta. Depois diga se está certa, o que faltou
```

```
   e qual conceito a justifica, em no máximo 5 linhas.
3. Se eu errar duas vezes no mesmo tema, explique com outro exemplo
   e pergunte de novo.
4. Use só estas regras de negócio e esta tabela de transições,
   sem inventar outras:
   <cole aqui RN01 a RN13 e a seção 4 do Mapa de Construção>
5. Não use Spring Boot 2 ou 3, javax.*, springdoc 2.x nem H2.

Comece pelo tema em que eu disser que estou mais fraco. Ao fim de
10 perguntas, liste os temas em que errei e o que devo reler.
```

O prompt desta semana: revisar o repositório sem alterar código

Use um assistente que enxerga o repositório da equipe, antes de criar a tag. O objetivo é um relatório, não uma correção: quem corrige é a equipe, para conseguir explicar a correção na apresentação.

```
Contexto: somos uma equipe construindo a API ServiceHub (Java 21,
Spring Boot 4.1 com starters por recurso, PostgreSQL 16, Flyway,
Springdoc 3.0.x). Vamos marcar a tag anotada etapa-2 (checkpoint da
1ª VA). O contrato oficial é o Mapa de Construção em docs/.

Objetivo: verificar o repositório contra o checklist abaixo e dizer,
item por item, o que está cumprido e o que falta.

Restrições:
- NÃO altere, crie nem apague nenhum arquivo. NÃO faça commit.
- NÃO escreva código de correção; descreva o que falta em uma frase.
- NÃO leia arquivos .env.
- Para cada item, responda CUMPRE, NÃO CUMPRE ou NÃO SEI, citando o
  arquivo (e a linha, se houver) que comprova. Sem evidência, a
  resposta é NÃO SEI: não suponha.

Checklist:
1. Migrações Flyway em src/main/resources/db/migration com as tabelas
   do domínio; spring.jpa.hibernate.ddl-auto=validate.
2. Nenhuma migração foi editada depois do commit que a criou
   (confira com git log de cada arquivo V*.sql).
3. Um CRUD completo em /api/v1, em camadas (controller, service,
   repository), com DTOs na entrada e na saída; nenhuma entidade
   JPA no corpo da requisição ou da resposta.
4. POST devolve 201 com o cabeçalho Location.
5. Listagem paginada: padrão 20 itens, máximo 100.
6. @RestControllerAdvice devolvendo 400 (com errors[]), 404 e 409 em
   ProblemDetail; nenhum throw new RuntimeException para regra.
7. OpenAPI: todas as rotas com @Operation e um @ApiResponse por
   status, com schema ProblemDetail nos erros.
8. Ao menos um @WebMvcTest; docs/dominio.md; ao menos um ADR.
9. README com: subir o Postgres, rodar a API, endereço do Swagger.
10. .env no .gitignore; nenhum segredo em arquivo versionado.
11. Se houver teste com banco, ele usa PostgreSQL, nunca H2.
```

```
Antes de verificar, descreva em até 5 linhas como vai checar cada
item (plano). Depois responda numa tabela:
item | situação | evidência | o que falta.

Critério de aceite: os 11 itens respondidos com evidência ou NÃO SEI,
e nenhum arquivo do repositório modificado.
```

O que a IA costuma errar nesta etapa

A IA sugere	No ServiceHub
"Use H2 para testar"	As migrações usam SQL do Postgres; Testcontainers (ADR-0005).
`ddl-auto=update`	O schema é do Flyway; use `validate`.
springdoc 2.x	Não funciona com o Boot 4; use a linha 3.0.x.
Editar uma V3 já aplicada	O checksum falha; crie uma nova versão.
`@MockBean`	Removido no Boot 4; use `@MockitoBean`.
`javax.persistence.*`	Desde o Boot 3 o pacote é `jakarta.persistence.*`.
"Corrigi os itens que faltavam"	Você pediu um relatório. Desfaça e corrija vocês mesmos.

Quase todos esses erros vêm de exemplos antigos da internet. Diga sempre a versão do Spring Boot no contexto do pedido.

Checklist de revisão

☐ Cada correção que a IA fez nas suas respostas de estudo bate com esta apostila ou com o Mapa?

☐ No relatório do repositório, cada CUMPRE tem um arquivo que você abriu e conferiu?

☐ Os itens NÃO SEI foram verificados por alguém da equipe?

☐ O `pom.xml` e o `application.properties` passam pela tabela de armadilhas acima?

☐ Você guardou um caso em que a IA errou e como descobriu o erro? Ele entra na entrega.

Prova

Duas provas, uma para cada uso. Para a revisão: depois da conversa com o assistente, `git status` mostra a árvore limpa, sem nenhum arquivo modificado. Para a entrega: no clone limpo, `git checkout etapa-2` seguido de `./mvnw verify` termina com `BUILD SUCCESS`.

15. Glossário

↑ conteúdo

Restrição REST	Regra do estilo arquitetural de Fielding, como "sem estado" ou "interface uniforme".
Idempotente	Operação que, repetida, deixa o servidor no mesmo estado que uma execução só.
Método seguro	Método HTTP que não altera o estado do servidor, como GET.
Máquina de estados	Modelo com estados possíveis e transições permitidas entre eles.
IoC	Inversão de controle: o framework cria e liga os objetos.
ORM	Mapeamento entre objetos do programa e tabelas do banco relacional.
Contexto de persistência	Conjunto de entidades managed que o Hibernate acompanha numa transação.
Dirty checking	Detecção automática de alterações em entidades managed, gravadas no commit.
ACID	Atomicidade, consistência, isolamento e durabilidade: garantias de uma transação.
Rollback	Desfazer todas as operações de uma transação que falhou.
Migração	Script versionado que leva o schema de uma versão à próxima.
Checksum	Soma de verificação do conteúdo de um arquivo; detecta edição.
Testcontainers	Biblioteca que sobe containers Docker reais durante os testes.
DTO	Objeto sem regra que carrega dados na fronteira da API.
ProblemDetail	Formato padrão de erro em JSON definido pela RFC 9457.
Tag anotada	Marca do Git com autor, data e mensagem, apontada para um commit.

16. Exercícios estilo prova

↑ conteúdo

Resolva sem consulta e com tempo marcado. Em cada resposta, diga o quê, o porquê e um caso do ServiceHub. Os exercícios 1 e 2 entram no `docs/revisao.md` da entrega.

1. Para cada cenário, dê o status code e uma frase que cite a regra ou o motivo: (a) `POST /api/v1/services` com `price = 0`; (b) um visitante pede `GET` de um serviço desativado; (c) o prestador muda uma ordem COMPLETED para IN_PROGRESS; (d) cadastro com `MARIA@X.COM` quando `maria@x.com` já existe; (e) `POST /api/v1/services` com `Content-Type: text/plain`; (f) `GET /api/v1/services?size=500`.

2. Ache pelo menos quatro problemas no código abaixo. Para cada um, diga o que está errado, o que o cliente da API sofre e como corrigir, indicando a camada.

```
@RestController
@RequestMapping("/servicos/criar")
public class ServicoController {
  @Autowired ServicoRepository repo;
  @PostMapping
  public Servico criar(@RequestBody Servico s) {
    if (s.getPrice().signum() <= 0)
      throw new RuntimeException("preço inválido");
    return repo.save(s);
  }
}
```

3. Verdadeiro ou falso? Nas falsas, reescreva a frase para que fique verdadeira. (a) PUT é idempotente; POST não é. (b) 409 indica que o JSON enviado está malformado. (c) Com `ddl-auto=validate`, o Hibernate cria as colunas que faltam. (d) Uma migração já aplicada pode ser editada para corrigir um erro. (e) `@Transactional` desfaz a transação quando o método lança `RuntimeException`. (f) `info.version`, no OpenAPI, é a versão da especificação OpenAPI usada. (g) Guardar a sessão do usuário na memória do servidor respeita o REST.

4. Associe cada conceito (A: DTO, B: Flyway, C: ProblemDetail, D: injeção de dependência, E: transação, F: OpenAPI) ao problema que ele resolve: (1) o front-end precisa conhecer campos e status sem ler o código; (2) desativar o usuário e os serviços dele, tudo ou nada; (3) a resposta estava expondo `passwordHash`; (4) cada erro da API vinha num formato diferente; (5) bancos de colegas com colunas diferentes; (6) não dá para testar o Service sem um banco ligado.

5. Considere o método abaixo, num `@Service`. Responda usando os termos managed, detached, commit e rollback: (a) o título muda no banco? (b) e se o método não tivesse `@Transactional`? (c) e se, depois do `setTitle`, o método lançasse uma `RuntimeException`?

```
@Transactional
public void renomear(Long id, String titulo) {
  ServiceOffering s = repo.findById(id).orElseThrow();
  s.setTitle(titulo);
  // não há repo.save(s)
}
```

6. Um cliente envia `DELETE /api/v1/services/12` duas vezes, por causa de uma falha de rede. A primeira resposta é 204. Explique por que o DELETE continua sendo idempotente, qualquer que seja a segunda resposta.

7. Uma equipe guarda o usuário logado numa variável de sessão na memória da API. Em produção, há duas instâncias atrás de um balanceador. Descreva o defeito que os usuários vão perceber e diga qual restrição REST foi violada.

8. Onde deve ficar a regra "ninguém contrata o próprio serviço" (RN06)? Justifique com dois argumentos e diga o status devolvido quando ela é violada.
9. JPA e Hibernate são a mesma coisa? Explique a relação entre os dois e cite o nome do problema que o ORM resolve.
10. O enum `RequestStatus` já impede, no Java, um status inválido. Por que a migração V5 ainda tem um `CHECK (status IN (...))`?
11. Um colega percebeu um erro de digitação no nome de uma coluna na `V3`, que já foi aplicada em todas as máquinas, e corrigiu o arquivo. O que acontece na próxima vez que a aplicação subir? Qual é o procedimento correto?
12. Um teste passa usando H2 e o deploy quebra no Postgres. O que o teste provou? Cite dois recursos de SQL das migrações do ServiceHub que motivam o uso de Testcontainers.
13. Um controller recebe a entidade `ServiceRequest` diretamente no `@RequestBody`. Que campos o cliente passaria a controlar? Que regras do Mapa ficam em risco?
14. Por que o cliente de uma API deve programar contra o campo `type` do ProblemDetail, e não contra o texto de `detail`?
15. Explique a diferença entre os cabeçalhos `Content-Type` e `Accept`. Qual status a API devolve quando não aceita o `Content-Type` enviado?
16. Projete método, URI e status de sucesso para: (a) o prestador aceitar a ordem 31; (b) listar, como prestador, as ordens PENDING, 20 por página; (c) o cliente avaliar a ordem 31 depois de concluída.
17. Por que a transação fica no Service, e não no Controller nem no Repository? Para que serve `readOnly = true`?
18. Cite três problemas que o teste no clone limpo (`git clone` + `git checkout etapa-2` + `./mvnw verify`) detecta e que passariam despercebidos na máquina de quem desenvolveu.
19. Escreva um prompt de tutor para estudar transações e o ciclo de vida da entidade. Ele deve ter contexto, a regra de não dar respostas antes da sua tentativa, cenários do ServiceHub e ao menos uma restrição de versão.
20. Um assistente respondeu: "para os testes, use H2 com `ddl-auto=update` e anote o mock com `@MockBean`". Aponte os três problemas dessa sugestão no ServiceHub e o que usar no lugar.

17. Para estudar

↑ conteúdo

Esta é uma semana de revisão: as leituras são as das semanas 1 a 5, disponíveis na Minha Biblioteca pelo acesso institucional. Se o tempo for curto, priorize os capítulos de *Integração de aplicações* e o capítulo 15 de Fowler.

Semana	Leitura
1 · APIs	RODRIGUES, T. N. et al. *Integração de aplicações*. Grupo A, 2020: "Tecnologias de integração entre sistemas e aplicações" e "SOAP" · SCHILDT, H. *Java para iniciantes*, cap. 1 · HORSTMANN, C. *Conceitos de computação com Java*, cap. 1 · RICHARDS, M.; FORD, N. *Fundamentos da arquitetura de software*, cap. 1
2 · REST	*Integração de aplicações*: "Geração de serviços com REST" e "Serviços web" · KHONONOV, V. *Aprenda Domain-Driven Design*, caps. 1–3 · PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*, caps. 7–8
3 · Spring	SCHILDT, caps. 8 e 12 · FOWLER, M. *Padrões de arquitetura de aplicações corporativas*, cap. 1 · MARTIN, R. C. *Arquitetura limpa*, caps. 22 e 32 · MONTEIRO, E. R. et al. *DevOps*: "Tipos de controles de versão" e "Criação de branches e commits com o Git" · MARTIN, R. C. *Código limpo*, caps. 2–3
4 · JPA	*Integração de aplicações*: "Servidores de aplicação JPA" · MACHADO, R. P. et al. *Desenvolvimento de software III*, cap. 4 · FOWLER, caps. 3 e 10 · SCHILDT, cap. 9 · *Código limpo*, cap. 7
5 · Contrato	*Integração de aplicações*: "Software de apoio" e "Conceito e uso de web service e APIs" · FOWLER, cap. 15 (Data Transfer Object)

Referências abertas:

- FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese (Doutorado), University of California, Irvine, 2000. Capítulo 5.
- RFC 9110, *HTTP Semantics* (métodos, propriedades seguro e idempotente, status codes): `rfc-editor.org/rfc/rfc9110`.
- RFC 9457, *Problem Details for HTTP APIs*: `rfc-editor.org/rfc/rfc9457`.
- Documentação oficial do Flyway, do Spring Boot (`docs.spring.io/spring-boot`) e do Testcontainers (`testcontainers.com`).
- Mapa de Construção do ServiceHub, em `docs/alunos`: o contrato oficial. Releia antes da prova.